

Heterogeneous Cognitive Transformers: Computational and Epistemic Coprocessors

Working Draft

August 2026

Abstract

Calculators, formal solvers, databases, and search engines are often grouped under tool use, although they repair different deficits and need not share an optimal interface. We present an evidence-aware position and research program in which a Transformer remains the learned semantic and generative substrate while computational coprocessors address competence deficits and epistemic coprocessors address knowledge deficits. A common runtime can recognize an emerging need, normalize it into a typed operation, invoke a bounded engine, preserve provenance-bearing state, and selectively materialize the result. Explicit tools and upfront retrieval remain general baselines and fallbacks. The stable model-facing contract is deliberately coarse: NONE, COMPUTE, RETRIEVE, VERIFY, or HELP. A deployment-specific runtime resolves that intent to a capability and implementation. Engine-specific tokens, adapters, fixed registries, and Progressive Retrieval Attention (PRA) are optional deployment choices rather than defining requirements. Developmental pilots expose the central challenge: calculator execution and controlled retrieval were reliable, while exposure, selection, normalization, timing, and result use determined end-to-end behavior. We therefore replace a single march toward tighter coupling with capability-specific interfaces, an eight-rung coupling ladder that includes semantic execution blocks, and evidence gates based on resolving interface questions. The paper contributes a modern Transformer-era synthesis, not a claim to have invented heterogeneous cognition, active retrieval, symbolic reasoning, or tool use. It states a claim hierarchy, failure taxonomy, roadmap, and broad falsification conditions for deciding whether this organization earns its complexity.

1 Introduction

Transformers are powerful semantic interfaces but awkward universal substrates. Exact arithmetic, logical closure, constraint propagation, source lookup, and freshness-sensitive verification already have specialized algorithms and data structures. Asking a language model to approximate every such operation internally can waste capacity and conceal uncertainty. Asking it to acquire every fact parametrically creates a different problem: knowledge can be absent, stale, private, source-specific, or contested.

Explicit tool calling is an important solution and should not be discarded. Yet “tool use” covers interfaces with different timing, state, and control semantics: a model can author an API call, a planner can schedule a solver, a retriever can run before generation, a control token can request evidence, or a runtime can recognize a bounded operation in developing text. The systems question is broader than whether tools should be automatic:

Which specialized computational or epistemic substrate can assist the evolving neural computation now, through what interface, and at what coupling level?

We call the design space *heterogeneous cognitive Transformers*. Here, “cognitive” has a narrow operational meaning concerning state participation in ongoing inference; it makes no claim about

Table 1: Competence and knowledge deficits imply different requests, success criteria, and characteristic risks.

	Competence deficit	Knowledge deficit	Composed case
Need	Perform or verify an operation	Acquire or verify evidence	Acquire inputs, then compute
Request	Typed expression, constraints, graph, program	Entity, relation, source, freshness	Evidence request plus typed operation
Output	Value, proof status, derivation, counterexample	Record, passage, provenance, evidence status	Provenance-linked derived result
Success	Formalization and execution correctness	Relevance, support, freshness, source quality	Both tests and correct composition
Primary risk	Wrong formalization or unsafe execution	Missing, stale, conflicting, or poisoned evidence	Laundering weak evidence through exact computation

consciousness or biological equivalence. The Transformer remains the general learned semantic and generative substrate. Specialized components recognize opportunities for assistance, normalize typed requests, invoke bounded engines, preserve provenance, materialize relevant outcomes, and abstain or fall back when an automatic interface is uncertain, costly, unsafe, or unsupported.

The central thesis is architectural and falsifiable. A useful runtime must separate deficits, expose boundary failures, and justify tighter coupling against strong simple alternatives. Early program evidence is included not as confirmation but because negative and mixed results have already changed the proposed interfaces and evidence gates. In particular:

- computational and epistemic coprocessors form twin tracks with a common interrupt, typed-state, and fallback contract;
- different capabilities may require different interfaces rather than a universal automatic reflex;
- semantic execution blocks provide an intermediate interface between verbose tool calls and implicit interrupts; and
- later mechanisms are justified when an earlier interface question is sufficiently resolved, not only when every earlier mechanism wins.

2 Two Deficits, Two Coprocessor Families

2.1 Competence and knowledge

A *competence deficit* occurs when the premises or operands are present, but a specialized operation would be more reliable or efficient than neural continuation. Arithmetic, type checking, graph reachability, logical inference, theorem proving, database aggregation, and bounded simulation are examples. A computational coprocessor returns a value, derivation, certificate, counterexample, or explicit failure under declared semantics.

A *knowledge deficit* occurs when a pending commitment needs information that is missing, stale, source-specific, private, or insufficiently supported. An epistemic coprocessor returns records, passages, source metadata, timestamps, conflicts, or an explicit no-evidence status. Retrieval provides evidence rather than truth: a precise retrieved string may still be stale or untrustworthy.

The distinction is operational rather than ontological. “Which region grew fastest?” can require both a versioned data lookup and exact aggregation. The separation remains useful because it localizes error and prevents an exact calculation from strengthening weak source provenance.

Table 2: Retrieval and program interfaces. Entries describe primary mechanisms, not every possible extension or deployment.

Dimension	Conventional RAG	FLARE	Self-RAG	Proposed program
Timing	Usually before generation	During generation	Adaptive during generation	Before or during generation
Control signal	Request or harness	Low confidence in predicted text	Learned retrieval/reflection tokens	Typed semantic event; later learned
Controller location	Orchestrator	External confidence logic	Main LM	Sidcar/runtime or main LM, compared
External family	Retriever	Retriever	Retriever	Compute and heterogeneous retrieval
State	Retrieved context	Evidence for regeneration	Evidence plus reflection signals	Typed provenance-bearing micro-state
Fallback	Deployment-specific	Continue/regenerate policy	Learned policy	Explicit tools and upfront RAG
Central question	What context helps?	When does confidence justify retrieval?	Can the LM learn retrieval/reflection?	Which substrate, interface, and coupling now?

2.2 Uncertainty and epistemic risk

Low next-token confidence is an observable model property. Epistemic risk is a property of a pending commitment’s dependence on external, fresh, source-specific, or verifiable evidence. A model can be uncertain about a timeless paraphrase and confident about a stale fact. Confidence can therefore be one retrieval trigger without being a complete account of evidence need.

3 Relationship to Adaptive Retrieval

Retrieval-augmented generation supplies non-parametric evidence to a generator [4]. FLARE is a direct precedent for generation-time retrieval: it predicts upcoming content and uses low-confidence tokens as a trigger [5]. Self-RAG trains the main LM to emit retrieval and reflection control tokens concerning relevance, support, and utility [6]. Adaptive-RAG, DRAGIN, corrective RAG, and multi-source retrieval further study when to retrieve, information needs, evidence correction, and source selection [7, 8, 9, 10]. Learned adaptive retrieval, support or relevance tokens, and generation-time retrieval are not novelty claims here.

Our narrower questions are whether semantic epistemic risk complements neural uncertainty, whether routing should span structurally different evidence substrates, whether control must live in the main LM, and whether a shared runtime contract can extend from retrieval to computation. Sidcar detectors may be model independent; learned control in the main model may ultimately be better. Both are hypotheses to compare.

4 Twin Architecture and Common Contract

Let $y_{1:t}$ denote the generated prefix and h_t an optional decoding state. A controller detects a pending need and may abstain:

$$d_t = D(y_{1:t}, h_t, \mathcal{S}_t), \quad d_t \in \{\text{NONE}, \text{COMPUTE}, \text{RETRIEVE}, \text{VERIFY}, \text{HELP}\}.$$

Normalization produces a typed event $e_t = N(y_{1:t}, h_t, \mathcal{S}_t)$ containing the family, operation or information need, arguments, scope, confidence, and budget. A router selects an engine $k = R(e_t)$ or abstains. Execution returns $z_t = E_k(e_t)$; the state manager records an auditable update

$$\mathcal{S}_{t+1} = U(\mathcal{S}_t, e_t, k, z_t, \pi_t),$$

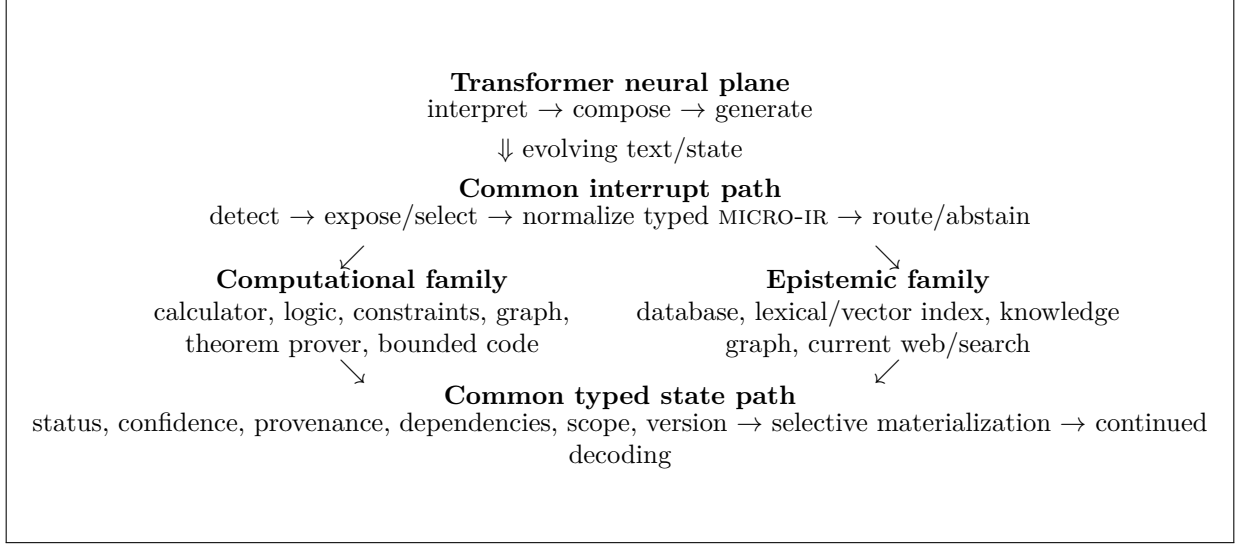


Figure 1: Twin computational and epistemic planes share interrupt and state contracts while retaining family-specific semantics and trust policies.

where π_t captures provenance, versions, timing, and dependencies. A materializer exposes only relevant state as decoding continues. Explicit tool calls can enter this same execution and state path.

4.1 Stable cognitive intent and dynamic capability resolution

The model need not identify an engine. We factor routing into

$$R_1 : (y_{1:t}, h_t, \mathcal{S}_t) \rightarrow \{\text{NONE}, \text{COMPUTE}, \text{RETRIEVE}, \text{VERIFY}, \text{HELP}\}$$

and

$$R_2 : (d_t, \text{task}, \text{registry}, \text{policy}, \mathcal{S}_t) \rightarrow \text{capability or shortlist}.$$

COMPUTE requests deterministic or structured work such as arithmetic, dates, units, graph closure, logic, algebra, or analytics. RETRIEVE requests external evidence or data. VERIFY asks the runtime to check an existing candidate using evidence, recomputation, logic, provenance, or active state. HELP is a fail-safe cognitive page fault: assistance is needed, but the model cannot reliably choose a family. NONE preserves ordinary generation.

R_1 is small and intended to remain stable across deployments. R_2 may use deterministic rules, lexical or token-aware BM25, CPU classifiers, capability retrieval, optional learned routing, or conventional tools. It selects semantic capabilities—for example arithmetic, Horn entailment, structured aggregate, or current factual lookup—rather than treating one backend name as the task. The runtime then chooses an implementation using exactness, available state, latency, cost, policy, credentials, and provenance requirements. Adding an engine should therefore not require retraining R_1 .

Three surface syntaxes remain empirical alternatives. Paired tags such as `<COMPUTE>` payload `</COMPUTE>` provide explicit incremental boundaries. A fenced COMPUTE block is familiar and potentially shorter but can collide with ordinary Markdown. A label-only form emits only the intent and lets the runtime reuse the active task as payload. Papers 2 and 2.5 measure tokenizer cost, parse reliability, and accidental activation rather than declaring a universal winner here.

4.2 Interoperable deployment modes

The architecture admits the modes in Table 3. They can be composed; they are not incompatible product editions.

Table 3: Deployment modes share the coarse intent and typed-state contracts.

Mode	Model/runtime boundary
Runtime-trigger coprocessor	No model modification; transparent detector emits intent.
In-context coprocessor	A short prompt teaches the generic-intent contract.
Generic adapter	Optional LoRA/SFT accelerates intent and result consumption.
PRA + in-context	PRA materializes only relevant capability or skill interfaces.
PRA + adapter	Learned coarse intent combines with dynamic discovery and state.
PRA only	Progressive materialization without a learned intent adapter.
Coprocessor only	Conventional registry insertion without PRA.
Conventional tools	Universal explicit fallback and matched baseline.

4.3 Three knowledge layers and optional PRA

The design separates (1) stable cognitive intent, which may live in a prompt, adapter, or native model; (2) dynamic capability knowledge, which belongs to the runtime registry, skill system, or PRA resources; and (3) cognitive state, including typed results, evidence, provenance, dependencies, and task scope. Conflating these layers would bake a changing product registry into model weights.

PRA is optional and orthogonal. Without PRA, the registry remains outside the prompt and the runtime inserts selected capability metadata conventionally. With PRA, capability descriptions and prior cognitive records can remain persistent but unmaterialized; task-aware retrieval exposes only a relevant shortlist and suppresses unrelated historical state. Thus a large registry need not imply a large active context. An adapter is likewise an optional accelerator for coarse intent, not the registry itself; engine-specific adapters remain possible optimizations, never a prerequisite for adding a coprocessor.

4.4 Operational cognitive state

We use a technical criterion rather than an anthropomorphic one:

Runtime state is treated as cognitive state when it persists beyond an isolated service response and can participate in subsequent interpretation, reasoning, selection, verification, or generation.

A one-shot observation displayed and discarded is an ordinary tool result. State that satisfies the criterion is typed, provenance-bearing, and potentially persistent. A minimal record contains a stable ID, type, payload, engine or source version, epistemic status, confidence, scope, dependencies, provenance, and timestamp. Computational state should retain the formalization and execution status, not only a number. Epistemic state should distinguish supported, contradicted, unverified, stale, ambiguous, and conflicting evidence. Persistence does not imply permanence: later work must support scope, retraction, and invalidation.

4.5 Unified boundary failures

Engine accuracy alone does not establish system accuracy. Every intervention should be attributed to a common failure taxonomy:

- **Detection:** a useful interrupt is missed or a false interrupt fires.
- **Exposure:** the relevant operation is never represented in usable form.
- **Selection:** the wrong candidate, span, or subexpression is chosen.
- **Normalization:** intended meaning is mapped incorrectly to MICRO-IR.

Table 4: Capability-specific interface hypotheses, risks, and retained fallbacks.

Capability	Likely interface	Main failure risk	Fallback
Calculator	Strict grammar or semantic block	Premature or partial subexpression	Explicit calculator tool
Logic	Typed block/parser	Incorrect formalization	Explicit symbolic request
Retrieval	Epistemic interrupt	False or missed evidence need	Explicit search/upfront RAG
Web/current facts	Semantic plus temporal trigger	Staleness or source conflict	Explicit search with sources
Code	Fenced executable block	Unsafe or unbounded execution	Sandboxed tool call
High-risk action	Explicit invocation	Unintended side effect	User confirmation

- **Routing:** the wrong engine or evidence source is selected.
- **Execution/retrieval:** execution fails or evidence is stale, irrelevant, conflicting, missing, or late.
- **State update:** provenance, dependencies, scope, or freshness are lost.
- **Materialization:** too much, too little, or the wrong state is reinjected.
- **Use:** correct output is available but ignored or misapplied.
- **Override:** later generation contradicts correct active state.
- **Safety:** execution or an external action violates its authorization.

The difficult part of heterogeneous cognitive systems is often the neural–coprocessor boundary, not the specialized engine itself.

5 Capability-Specific Coupling

Different coprocessor families may have different optimal interfaces.

This is a design hypothesis, not an established result. A calculator may benefit from a narrow grammar or typed block; open-web search may require a semantic and temporal trigger; consequential actions should remain explicit. Interface choice should follow ambiguity, reversibility, latency, semantic surface area, and harm rather than a global preference for automaticity.

5.1 Semantic execution blocks

Semantic blocks explicitly delimit a machine-executable cognitive region without requiring the model to author a verbose API schema. Examples are:

```
```calculator
15246377 * 746647383
```
```

```
```datalog
fact isa(penguin,bird)
fact isa(bird,animal)
query isa(penguin,animal)
```

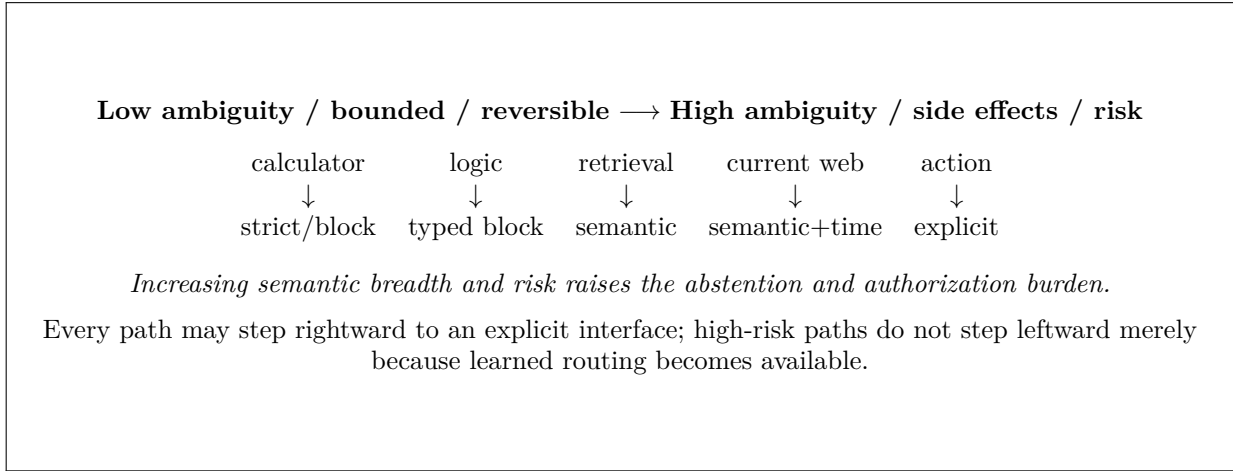


Figure 2: A capability-specific coupling map. Position is a testable design hypothesis, not a maturity ranking.

```

...

```python
# bounded sandboxed code
...

```

The runtime detects the opening block type, waits for the closing delimiter, normalizes the enclosed content, executes it with a corresponding bounded local engine, returns a typed result, and then continues decoding. Waiting for the close avoids the premature-subexpression problem of streaming lexical reflexes. Typed blocks also expose what the model intended to offload and permit syntax, resource, and capability checks before execution. Arbitrary code belongs in a no-network, resource-bounded sandbox; domain-specific engines are preferable when they suffice.

5.2 An eight-rung coupling ladder

Coupling is a set of alternatives to compare, not a one-way definition of progress. Self-RAG-style special tokens provide a precedent for learned control signals; semantic execution blocks generalize explicit typed control across computational families without requiring learned tokens. Neither those blocks nor capability-specific tokens define a cognitive coprocessor: they are candidate surfaces beneath the stable R_1/R_2 factorization.

The operational fallback hierarchy runs in the other direction when confidence or safety declines: Level A uses cheap automatic reflexes; Level B uses bounded semantic blocks; Level C uses automatic semantic or epistemic interrupts; Level D adds learned routing; Level E uses explicit tools for arbitrary, ambiguous, expensive, or side-effectful capabilities; and Level F requires user confirmation for high-risk actions. Explicit tools remain the universal slow, general fallback.

6 Early Evidence from the Program

The following results are preliminary developmental evidence. Protocols are small, one retrieval protocol was tuned during development, and neither pilot establishes model-family or task-family generality. Their role here is to show how evidence changed the architecture.

- | | | |
|---|---|------|
| 1 | Explicit tool or function call: universal, inspectable baseline. | |
| 2 | Strict syntax/grammar reflex: narrow deterministic recognition. | |
| 3 | Semantic execution block: explicit typed fenced region. | |
| 4 | Lexical/syntactic semantic interrupt: runtime recognizes ordinary text. | |
| 5 | Heuristic routing: transparent policy selects engines or sources. | more |
| 6 | Learned text-level interrupt: calibrated detector/parser with abstention. | |
| 7 | Hidden-state routing: frozen neural state informs intervention. | |
| 8 | Jointly trained/native interface: model and runtime co-adapt. | |
| | implicit and co-adapted $\longrightarrow$ | |
| | <i>Advance only for a measured boundary failure or matched benefit.</i> | |

Figure 3: Revised coupling ladder. A higher rung is not inherently better; cost, observability, calibration, and safety remain evaluation dimensions.

6.1 What the pilots do and do not show

In Paper 1’s specified strict-reflex pilot, the calculator was exact whenever invoked, but only roughly half of accepted candidates represented the intended full expression, and correct outputs could be ignored or overridden. Automatic invocation therefore did not automatically outperform an explicit calculator. The harder held-out extension suggests that normalization can matter, while also showing that a superficially attractive block interface can fail if the generation contract does not reliably close and expose executable content.

Paper 1.5 provides a more promising early signal for retrieval coupling than the strict arithmetic reflex. Its one distinguishing stale case supports the possibility that semantic risk complements confidence; it does not demonstrate general superiority over FLARE. The protocol was tuned during development, the source was controlled, and upfront RAG matched accuracy at greater retrieval cost. Negative outcomes and strong fallback performance are part of the evidence, not inconveniences to omit.

Together, the studies motivate measuring detection, exposure, selection, normalization, materialization, use, and override separately. Specialized engines can be correct while the heterogeneous system is not.

7 Program Claims and Falsification

7.1 Claim hierarchy

The program separates increasingly ambitious claims:

Claim A. Specialized engines are often more reliable or efficient for exact operations or external evidence.

Claim B. Automatic generation time coupling can reduce explicit invocation burden in selected regimes.

Table 5: Developmental evidence from Papers 1 and 1.5. Percentages are exact accuracy unless stated otherwise; no row is a confirmatory general claim.

Study	Observed result	Boundary diagnosis	Program consequence
Paper 1 strict pilot	Qwen3-0.6B XPU: LLM-only 75%; strict reflex 75%; explicit calculator 87.5%; oracle 100%. Engine exact when invoked; about 50% of accepted candidates captured the intended full expression.	Exposure/selection and use/override failures, not calculator arithmetic, limited the reflex.	Falsifies “automatic reflex > tool call” in this easy regime; retain explicit tools and compare other interfaces.
Paper 1 hard held-out extension	Same checkpoint: LLM-only 43.75%; explicit 62.5%; strict 37.5%; normalized reflex 75%; blocks 12.5%; oracle 87.5% on 16 arithmetic cases. Normalized vs. LLM-only exact McNemar $p = 0.0625$; zero false interventions on 12 controls.	Normalization helped, but the block prompt exposed placeholder/closure failure and the oracle gap remained. Engine execution stayed exact.	Characterize and replicate the normalized interface; do not infer that blocks or automatic compute generally win.
Paper 1.5 retrieval pilot	Qwen3-0.6B, 12 controlled-source cases: FLARE-like confidence 83.3% accuracy at 83.3% retrieval; semantic OR confidence 91.7% at 91.7%. Upfront RAG also reached 91.7% while retrieving all cases.	One high-confidence/high-risk stale case drove the OR difference. Confidence over-retrieved three low-confidence/low-risk cases.	Uncertainty and epistemic risk were observably distinct in one controlled case; semantic risk may complement confidence. Frozen confirmation is still required.

Claim C. Persistent typed cognitive state improves multi-step reasoning and reuse.

Claim D. Learned semantic interrupts outperform hand-built routing.

Claim E. Native, key/value, or latent coupling improves over textual or structured materialization.

Claim F. Jointly trained heterogeneous systems develop a better division of labor than monolithic neural models.

Claim A is supported by the existence and observed correctness of bounded engines, but does not imply Claim B. The early pilots probe B in narrow regimes and cannot establish C–F. Each later claim needs its own matched baseline, cost model, ablation, and failure analysis.

7.2 What would falsify the broad program?

The broad program is weakened if, across multiple domains:

- explicit tool calling consistently matches or dominates automatic coupling on accuracy and cost;
- semantic interrupts create more false interventions than useful assistance;
- state and provenance overhead exceeds saved neural computation;
- heterogeneous routing provides no benefit over universal tools or retrievers;
- small models do not gain disproportionately;
- tighter interfaces do not improve scaling with task difficulty; or
- hidden or native coupling adds complexity without measurable value.

A successful program does not require every family to win. A valid outcome may be that some capabilities benefit from reflex coupling, others from execution blocks, others from learned routing, and high-risk actions remain explicit. Conversely, a sequence of narrow positive pilots would not validate a universal heterogeneous architecture without cross-domain and systems-level benefit.

8 Evidence-Gated Twin Roadmap

The governing rule is:

A later paper may proceed when the earlier interface question has been resolved sufficiently to motivate the next mechanism. Do not generalize an unresolved interface.

For Paper 1 to Paper 2, a strict reflex need not win; the computational interface must be sufficiently characterized. A successful strict reflex can be reused, a better semantic block can replace it, semantic-selection failure can make learned interrupts more urgent, and persistent explicit-tool superiority must narrow the automatic-compute claim. For Paper 1.5 to Paper 2.5, generation-time retrieval must have value in some regime and/or oracle source heterogeneity must show measurable headroom. Paper 2.5 motivates Paper 3.5 only if source selection matters and heuristics leave an oracle gap. Paper 4 and beyond require concrete provenance or retraction failures caused by persistent multi-engine or multi-source state.

9 Architectural Antecedents and Synthesis

The proposal synthesizes old and new systems ideas. Toolformer and function calling establish model-mediated APIs [1]; ReAct and planner-executor systems interleave reasoning and action [2]; PAL delegates generated programs to interpreters [3]. Logic-LM, AlphaGeometry, LeanDojo, and dynamic solver composition demonstrate neural formalization and symbolic or theorem-prover interaction [11, 12, 13, 14]. Neural module networks compose learned specialist modules [15].

Blackboard architectures organize heterogeneous knowledge sources around shared state and opportunistic control [16]; production systems and classical cognitive architectures provide antecedents for event-driven rules, persistent working state, and modular cognition [17, 18, 19]. Typed intermediate representations such as MLIR show how explicit contracts support heterogeneous execution [20]; capability-restricted WebAssembly illustrates a portable safe-execution substrate rather than a security proof by itself [21]. Heterogeneous computing supplies the coprocessor analogy: specialized substrates can coexist behind explicit scheduling, data movement, and consistency contracts. MemGPT and InferCept connect interruption and persistent state to modern LM runtime management [22, 23].

The synthesis differs from “tools, but automatic.” It treats interface type as capability-specific, makes provenance-bearing state a first-class but gated hypothesis, permits runtime and model control to compete, and requires automatic paths to abstain into explicit general fallbacks.

10 Evaluation Principles and Safety

Every experiment should compare LLM-only, matched-prompt, explicit tool or retrieval, automatic candidate interfaces, and an oracle boundary condition. Report end-to-end correctness alongside detector precision/recall, candidate coverage, selection and normalization accuracy, engine correctness, source quality, materialization/use/override rates, latency, tokens, calls, and state cost. Difficulty scaling and adversarial non-trigger controls are required. Sham interrupts can separate pause effects from result effects. Paired tests, confidence intervals, seeds, frozen protocols, and held-out cells should replace claims based on aggregate point differences.

Automatic execution should be bounded, local, reversible, observable, and side-effect free by default. Sandboxes need explicit CPU, memory, recursion, time, filesystem, and network policies. Retrieved material is data, not a control channel; prompt injection, source poisoning, stale evidence, and source conflicts must be represented rather than silently promoted to state. User confirmation remains mandatory for high-risk external actions. Provenance and state do not guarantee truth or safety, but they make dependencies inspectable and failures attributable.

Table 6: Twin roadmap and evidence question. A gate resolves an interface decision; it does not require every candidate mechanism to win.

Paper	Scope	Evidence question	Decision enabled
0	Position and architecture	Are deficits, interfaces, failures, and falsifiers operational?	Freeze a testable program, not an empirical conclusion.
1	Calculator/interface	Which strict, normalized, block, or explicit interface survives selection and use failures?	Choose a bounded compute interface or narrow automaticity.
1.5	Single-source epistemic retrieval	Does generation-time retrieval help in any regime; does semantic risk add signal beyond confidence?	Freeze trigger questions and quantify source-heterogeneity headroom.
2	Heterogeneous compute	Does generic COMPUTE/VERIFY/HELP survive registry expansion while runtime capability routing remains exact?	Prefer registry-independent intent; learn engine identity only for measured residual value.
2.5	Heterogeneous retrieval	Does generic RETRIEVE/VERIFY/HELP survive backend changes while source-native semantics beat matched universal retrieval?	Decouple model intent from the evolving data registry.
3	Learned compute interrupts	Can learned detection/selection beat transparent compute policies under shift?	Justify hidden-state or joint compute coupling.
3.5	Learned epistemic interrupts	Can learned trigger/source routing beat calibrated confidence and semantic heuristics?	Justify learned epistemic control.
4	Transactions, provenance, retraction	Does persistent multi-family state create concrete dependency, rollback, or conflict failures?	Add only the state machinery needed by observed failures.
5	Structured/PRA/native materialization	Does selective structured access improve over textual reinjection at matched cost?	Justify native or key/value interfaces.
6	Co-adaptation	Does joint training improve calibrated division of labor without losing fallback robustness?	Justify end-to-end adaptation.
7	Integrated runtime	Do combined capabilities improve reliability, cost, and scaling over strong modular agents?	Accept, narrow, or reject the integrated architecture.

Table 7: Expanded antecedent matrix. The proposed contribution is a unified, evidence-gated Transformer-era synthesis rather than invention of any row.

Antecedent	Primary idea	control/state	What it establishes	Question retained here
Tool/function calling	Model or orchestrator emits API request		General external capabilities	When should bounded needs avoid or fall back to calls?
ReAct/planner-executor	Interleaved authored plans/actions	model-	In-trajectory external action	Can runtime recognition help without a full authored plan?
PAL/program-aided LMs	Generated program interpreter	plus	Verifiable external execution	Which block/normalization contract is reliable?
FLARE/Self-RAG/adaptive retrieval	Confidence, learned kens, or adaptive policies	to-	Generation-time evidence control	Is semantic risk complementary; where should control live?
Neuro-symbolic, theorem, solver composition	Neural formalization plus formal engines		Specialized exact reasoning	How should selection and formalization failures be exposed?
Neural module networks	Learned composition of specialist modules	Task-dependent	modular computation	Does module routing transfer to open-ended generation?
Blackboard/shared-state systems	Opportunistic over common state	agents	Heterogeneous coordination	Which typed state and conflict semantics are necessary?
Production/cognitive architectures	Rules, events, working memory	modules,	Persistent modular control	Which ideas survive modern Transformer evaluation?
Heterogeneous compute	Specialized plus data movement	processors	Capability-specific substrates	What are the scheduling/materialization costs?
Typed IR/event-driven systems	Explicit events and tracts	con-	Inspectable routing and lowering	How much semantics can be normalized safely?
Safe execution sand-boxes	Capability and confinement	and resource	Bounded execution mechanisms	Which generated regions are safe enough to run?
LM runtime/memory systems	Interrupt and scheduling	state	Systems costs of augmented inference	When does persistent state earn its overhead?

11 Discussion and Limitations

The competence/knowledge distinction may blur in composed tasks, and its value must be judged by better error localization and experimental prediction. Interrupts can arrive too early to expose a complete operation or too late to prevent a commitment. Text may be a sufficient interface longer than expected; native state access may add latency and opacity without benefit. Persistent state can itself become a context and consistency burden. Small pilots on one checkpoint cannot forecast behavior for stronger models, long-form generation, open-web sources, or consequential domains.

Progressive Retrieval Attention is therefore not central before Paper 5. Explicit tools, upfront RAG, transparent policies, and abstention remain valid end states rather than temporary baselines destined for replacement.

If heterogeneous coprocessors become numerous and persistent, a later research direction can ask whether natural language should remain the universal internal interchange format, or whether typed/learned cognitive interlingua are more appropriate.

12 Conclusion

A Transformer need not be either a complete monolithic reasoner or a thin tool dispatcher. It can remain the learned semantic and generative substrate while bounded computational and epistemic coprocessors contribute exact operations and external evidence. The unifying research object is the boundary: how a need is exposed, selected, normalized, routed, recorded, materialized, used, and safely overridden or retracted.

Developmental Papers 1 and 1.5 sharpen rather than confirm the thesis. They show reliable engines behind fragile interfaces, mixed automatic results, useful explicit fallbacks, and one controlled distinction between uncertainty and epistemic risk. The resulting program is capability-specific, fallback-preserving, and evidence-gated. It succeeds only where heterogeneous coupling yields reproducible

reliability, efficiency, scaling, or small-model benefit over simpler alternatives; elsewhere, the correct architecture is the simpler one.

The preferred scalable boundary is therefore a coarse cognitive interrupt, deployment-specific runtime or PRA capability resolution, specialized execution, typed cognitive state, and selective continuation. LoRA may make that interrupt cheaper or more reliable, but it should not become a catalog of engines. Conventional tools remain the fallback when automatic intent or runtime resolution is uncertain.

References

- [1] T. Schick et al. Toolformer: Language models can teach themselves to use tools. *NeurIPS*, 2023. <https://arxiv.org/abs/2302.04761>.
- [2] S. Yao et al. ReAct: Synergizing reasoning and acting in language models. *ICLR*, 2023. <https://arxiv.org/abs/2210.03629>.
- [3] L. Gao et al. PAL: Program-aided language models. *ICML*, 2023. <https://arxiv.org/abs/2211.10435>.
- [4] P. Lewis et al. Retrieval-augmented generation for knowledge-intensive NLP tasks. *NeurIPS*, 2020. <https://arxiv.org/abs/2005.11401>.
- [5] Z. Jiang et al. Active retrieval augmented generation. *EMNLP*, 2023. <https://arxiv.org/abs/2305.06983>.
- [6] A. Asai, Z. Wu, Y. Wang, A. Sil, and H. Hajishirzi. Self-RAG: Learning to retrieve, generate, and critique through self-reflection. *ICLR*, 2024. <https://arxiv.org/abs/2310.11511>.
- [7] S. Jeong, J. Baek, S. Cho, S. J. Hwang, and J. C. Park. Adaptive-RAG: Learning to adapt retrieval-augmented language models through question complexity. arXiv:2403.14403, 2024.
- [8] W. Su, Y. Tang, Q. Ai, Z. Wu, and Y. Liu. DRAGIN: Dynamic retrieval augmented generation based on the information needs of large language models. arXiv:2403.10081, 2024.
- [9] S.-Q. Yan, J.-C. Gu, Y. Zhu, and Z.-H. Ling. Corrective retrieval augmented generation. arXiv:2401.15884, 2024.
- [10] Q. Zhao, R. Wang, X. Wang, D. Zha, and N. Mu. Towards multi-source retrieval-augmented generation via synergizing reasoning and preference-driven retrieval. arXiv:2411.00689, 2024.
- [11] L. Pan, A. Albalak, X. Wang, and W. Y. Wang. Logic-LM: Empowering large language models with symbolic solvers for faithful logical reasoning. *Findings of EMNLP*, 2023.
- [12] T. H. Trinh, Y. Wu, Q. V. Le, H. He, and T. Luong. Solving olympiad geometry without human demonstrations. *Nature*, 2024.
- [13] K. Yang et al. LeanDojo: Theorem proving with retrieval-augmented language models. *NeurIPS*, 2023.
- [14] L. Xu, P. Beckmann, M. Valentino, and A. Freitas. Adaptive LLM-symbolic reasoning via dynamic logical solver composition. arXiv:2510.06774, 2025.
- [15] J. Andreas, M. Rohrbach, T. Darrell, and D. Klein. Neural module networks. *CVPR*, 2016.

- [16] B. Hayes-Roth. A blackboard architecture for control. *Artificial Intelligence*, 26(3):251–321, 1985.
- [17] C. L. Forgy. Rete: A fast algorithm for the many pattern/many object pattern match problem. *Artificial Intelligence*, 19(1):17–37, 1982.
- [18] A. Newell. *Unified Theories of Cognition*. Harvard University Press, 1990.
- [19] J. R. Anderson et al. An integrated theory of the mind. *Psychological Review*, 111(4):1036–1060, 2004.
- [20] C. Lattner et al. MLIR: Scaling compiler infrastructure for domain specific computation. *CGO*, 2021.
- [21] A. Haas et al. Bringing the Web up to speed with WebAssembly. *PLDI*, 2017.
- [22] C. Packer et al. MemGPT: Towards LLMs as operating systems. arXiv:2310.08560, 2023.
- [23] R. Abhyankar, Z. He, V. Srivatsa, H. Zhang, and Y. Zhang. InferCept: Efficient intercept support for augmented large language model inference. arXiv:2402.01869, 2024.